



I E S A L B A R R E G A S

TÉCNICO SUPERIOR EN DESARROLLO DE
APLICACIONES WEB

Departamento de Informática

PROYECTO

Pista Libre
Manual Técnico

Autor/a: José María Núñez González
Curso Académico: 2025/202

Índice

1. Introducción	3
1.1 Descripción general del proyecto	3
1.2 Funcionalidades implementadas	3
2. Arquitectura de la aplicación	4
2.1 Frontend.....	5
2.1.1 Tecnologías usadas	5
2.1.2 Entorno de desarrollo.....	6
2.2 Backend	7
2.2.1 Tecnologías usadas	7
2.2.2 Entorno de desarrollo.....	8
3. Documentación técnica.....	9
3.1. Análisis.....	9
3.2. Desarrollo (diagrama de secuencias...).....	13
3.3. Pruebas realizadas	16
4. Proceso de despliegue	19
4.1 Requisitos software.....	19
4.2 Descarga del proyecto	19
4.3 Configuración de la base de datos	20
4.4 Configuración de la conexión.....	20
4.5 Despliegue en Apache	21
4.6 Acceso a la aplicación	21
4.7 Usuarios de prueba.....	21
4.8 Hosting.....	22
5. Propuestas de mejoras	23
6. Bibliografía	24

1. Introducción

1.1 Descripción general del proyecto

PistaLibre es una aplicación web desarrollada para la gestión y reserva de instalaciones deportivas. El objetivo principal del proyecto es facilitar la consulta de disponibilidad y la reserva de pistas deportivas por parte de los usuarios registrados, así como proporcionar herramientas de gestión para profesores y administradores.

La aplicación permite gestionar diferentes centros deportivos, deportes y pistas, ofreciendo una solución centralizada para el control de reservas y actividades deportivas.

El sistema ha sido diseñado siguiendo una arquitectura cliente-servidor basada en tecnologías web estándar, utilizando una API REST desarrollada en PHP y una base de datos relacional MariaDB/MySQL.

1.2 Funcionalidades implementadas

Las principales funcionalidades desarrolladas son:

Gestión de usuarios

- Registro de nuevos usuarios.
- Inicio y cierre de sesión.
- Control de acceso mediante roles.
- Activación y desactivación de usuarios.

Gestión de reservas

- Consulta de disponibilidad de pistas.
- Creación de reservas deportivas.
- Cancelación de reservas.
- Visualización del historial de reservas.

Gestión de participantes

- Incorporación de acompañantes a una reserva.
- Eliminación de acompañantes.
- Control de participación en reservas.

Gestión de anotaciones

- Registro de resultados deportivos.
- Valoración de instalaciones.
- Comentarios sobre pistas y centros deportivos.
- Modificación posterior de anotaciones.

Funcionalidades para profesores

- Creación de reservas fijas periódicas.
- Liberación puntual de clases.
- Creación de clases de recuperación.
- Consulta detallada de clases programadas.

Funcionalidades para administradores

- Gestión de usuarios.
- Gestión de reservas fijas.
- Liberación de clases de cualquier profesor.
- Creación de recuperaciones.
- Desactivación de reservas fijas.

2. Arquitectura de la aplicación

La aplicación se ha desarrollado siguiendo una arquitectura cliente-servidor basada en tecnologías web estándar.

El sistema está dividido en dos grandes bloques:

- Frontend.
- Backend.

La comunicación entre ambos componentes se realiza mediante peticiones HTTP a una API REST desarrollada en PHP.

Estructura general del proyecto

```
pistalibre/  
├── backend/  
│   ├── config/  
│   ├── controllers/  
│   ├── models/  
│   └── routes/  
├── frontend/  
│   ├── assets/  
│   ├── css/  
│   ├── js/  
│   └── pages/  
├── .gitignore  
└── README.md
```

2.1 Frontend

2.1.1 Tecnologías usadas

HTML5

Se ha utilizado HTML5 para definir la estructura de todas las páginas de la aplicación.

Su elección se debe a que es el estándar actual para el desarrollo web y proporciona compatibilidad con todos los navegadores modernos.

CSS3

Se ha utilizado CSS3 para la maquetación y personalización visual de la aplicación.

Permite implementar:

- Diseño responsive.
- Flexbox.
- Grid Layout.
- Animaciones y transiciones.

JavaScript ES6

JavaScript constituye la base de toda la interacción dinámica de la aplicación.

Se utiliza para:

- Comunicación con la API REST.
- Validación de formularios.
- Gestión de eventos.
- Manipulación dinámica del DOM.
- Gestión de sesiones mediante LocalStorage.

Se ha utilizado sintaxis ES6 para mejorar la legibilidad y mantenibilidad del código.

Fetch API

Se ha utilizado la API Fetch nativa de JavaScript para realizar peticiones HTTP al backend.

Su utilización evita dependencias externas y proporciona una integración sencilla con los servicios REST desarrollados.

2.1.2 Entorno de desarrollo

Sistema operativo

Ubuntu 22.04.5 LTS

Editor de código

Visual Studio Code

Visual Studio Code fue seleccionado debido a:

- Amplio soporte para PHP, HTML, CSS y JavaScript.
- Sistema de extensiones.
- Integración con Git.
- Herramientas de depuración.

Control de versiones

Git

Permite mantener el historial de cambios del proyecto y facilita el desarrollo incremental.

Repositorio remoto

GitHub

El código fuente se encuentra almacenado y versionado mediante GitHub.

2.2 Backend

2.2.1 Tecnologías usadas

PHP

Versión utilizada:

PHP 8.1.2

PHP se ha utilizado para implementar toda la lógica de negocio y la API REST.

Las razones de su elección son:

- Integración nativa con Apache.
- Facilidad de conexión con bases de datos MySQL.
- Amplia documentación.
- Tecnología estudiada durante el ciclo formativo.

MySQL

Versión utilizada:

MySQL 8.0.45

La base de datos se ha diseñado siguiendo un modelo relacional normalizado.

Se eligió por:

- Alto rendimiento.
- Facilidad de administración.
- Compatibilidad con PHP mediante PDO.
- Amplio soporte documental.

PDO

Se ha utilizado PDO para todas las operaciones de acceso a datos.

Ventajas:

- Prevención de inyecciones SQL.
- Compatibilidad con diferentes motores de bases de datos.
- Gestión segura de consultas parametrizadas.

API REST

La aplicación implementa una arquitectura REST mediante endpoints independientes ubicados en:

backend/routes/

Algunos ejemplos son:

login.php

registro.php

disponibilidad.php

crear_reserva.php

cancelar_reserva.php

crear_anotacion.php

reservas_fijas.php

liberar_reserva_fija.php

2.2.2 Entorno de desarrollo

Sistema operativo

Ubuntu 22.04.5 LTS

Servidor web

Apache/2.4.52 (Ubuntu)

Apache ha sido utilizado para servir tanto el frontend como la API REST.

Herramientas de pruebas

Postman

Se utilizó para:

- Probar endpoints REST.
- Validar respuestas JSON.
- Comprobar errores y validaciones.
- Simular peticiones HTTP.

3. Documentación técnica

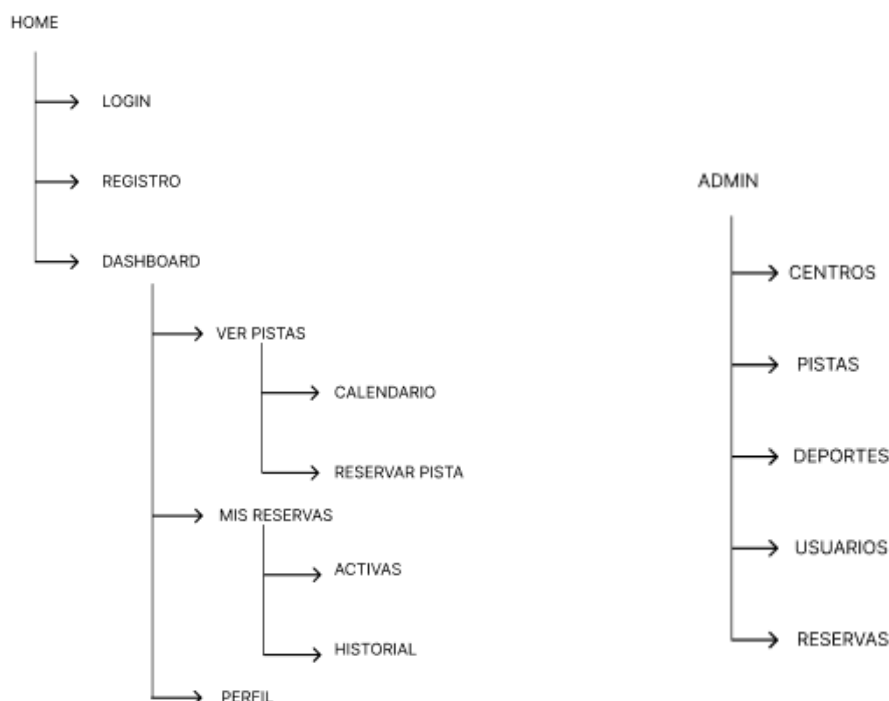
3.1. Análisis

Durante la fase de análisis se definieron los requisitos funcionales y la estructura general de la aplicación, identificando los diferentes tipos de usuarios, las funcionalidades necesarias y las entidades que formarían parte del sistema.

El análisis permitió diseñar una solución capaz de gestionar reservas deportivas, clases periódicas impartidas por profesores y funciones administrativas para el mantenimiento de la aplicación.

Mapa de navegación

La estructura funcional de la aplicación se definió mediante un mapa de navegación que muestra las diferentes páginas y opciones disponibles para los usuarios.



El mapa de navegación permite identificar los distintos recorridos disponibles dentro de la aplicación y facilita la organización de las interfaces de usuario.

Actores del sistema

Durante el análisis se identificaron tres perfiles principales de usuario:

Usuario

Puede:

- Registrarse.
- Iniciar sesión.
- Consultar disponibilidad.
- Crear reservas.
- Cancelar reservas.
- Gestionar acompañantes.
- Añadir anotaciones deportivas.

Profesor

Además de las funciones de usuario:

- Gestiona clases periódicas.
- Libera clases concretas.
- Programa recuperaciones.
- Consulta el estado de sus reservas fijas.

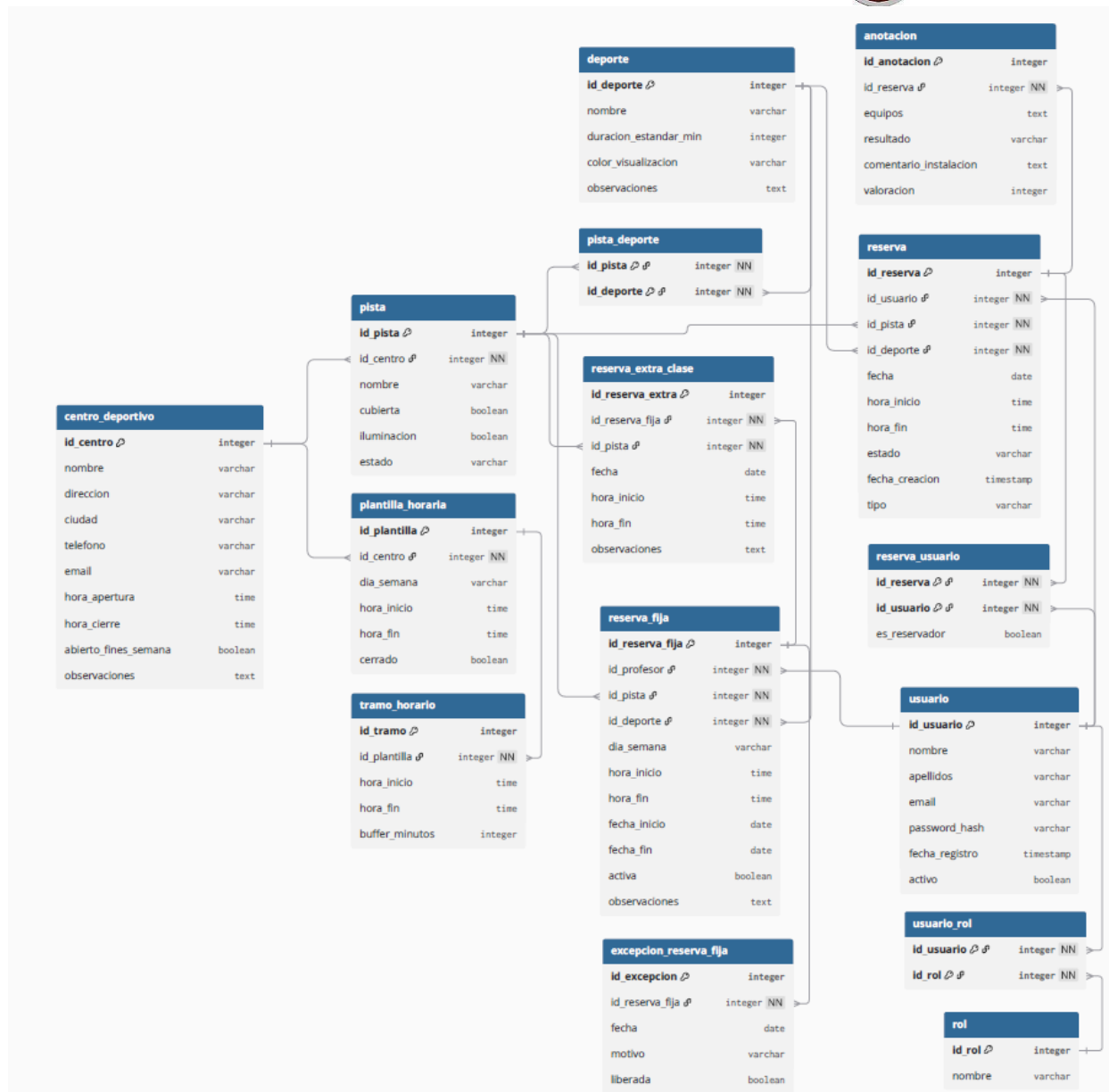
Administrador

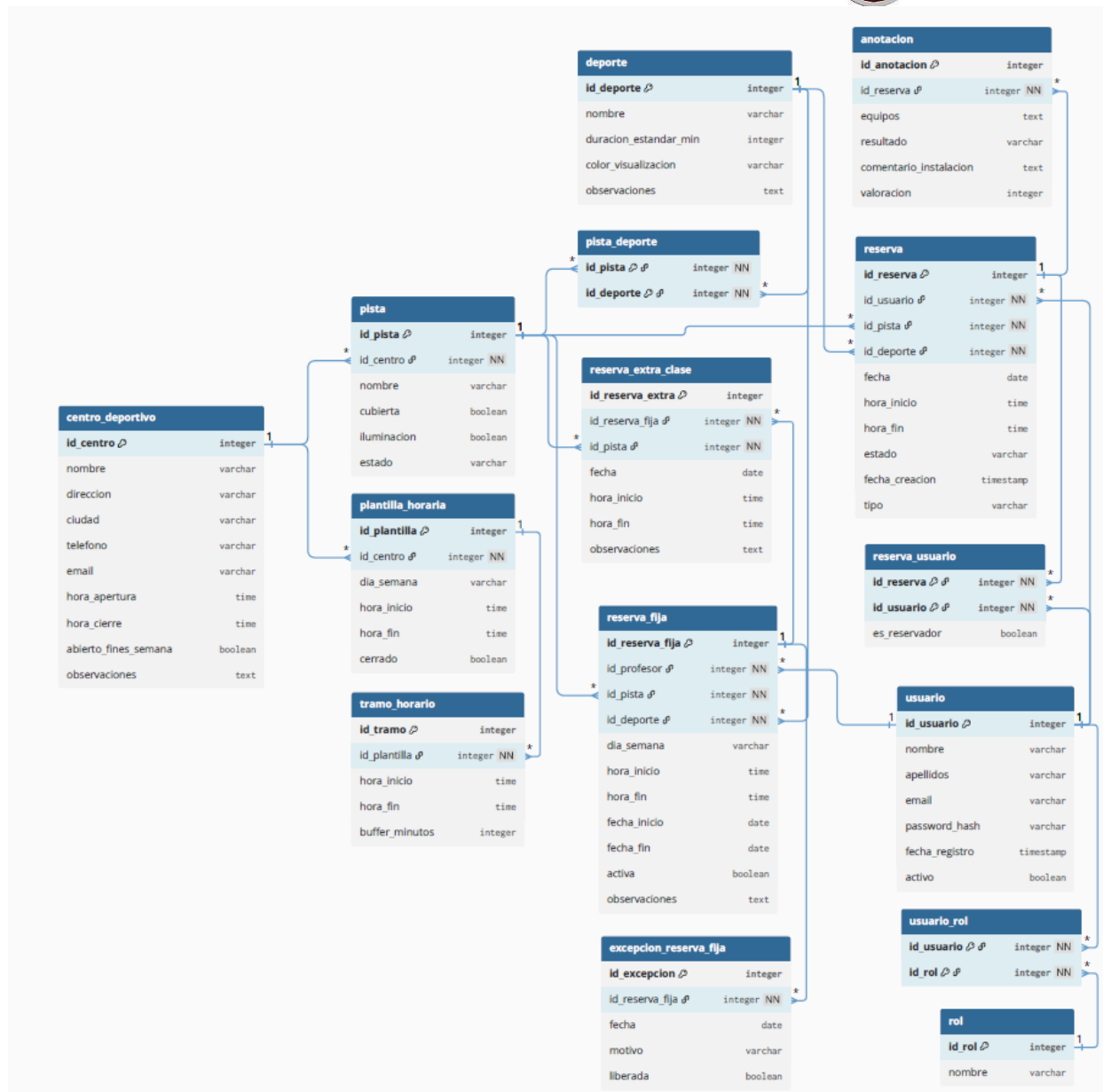
Dispone de acceso completo al sistema:

- Gestión de usuarios.
- Activación y desactivación de cuentas.
- Gestión de reservas fijas.
- Liberación de clases de cualquier profesor.
- Programación de recuperaciones.
- Supervisión general de la aplicación.

Modelo Entidad-Relación

A partir de los requisitos funcionales se diseñó un modelo Entidad-Relación que representa la estructura de la base de datos y las relaciones existentes entre las distintas entidades.





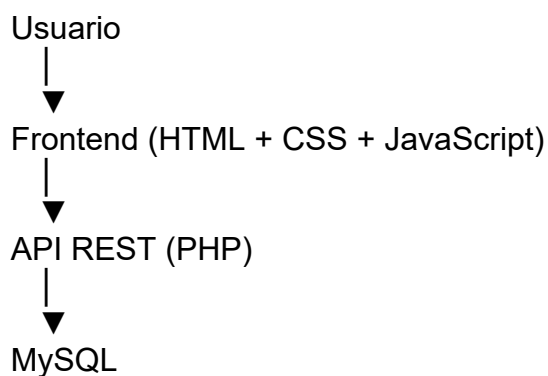
3.2. Desarrollo (diagrama de secuencias...)

Una vez finalizada la fase de análisis se procedió al desarrollo de la aplicación utilizando una arquitectura basada en API REST.

La comunicación entre el frontend y el backend se realiza mediante peticiones HTTP que intercambian información en formato JSON.

Arquitectura implementada

La arquitectura desarrollada puede representarse mediante el siguiente esquema:



El frontend se encarga de la interacción con el usuario, mientras que el backend implementa la lógica de negocio y el acceso a los datos almacenados en la base de datos.

Modelo lógico de datos

El modelo lógico implementado está compuesto por las siguientes tablas principales:

Gestión de usuarios

- usuario
- rol
- usuario_rol

Permiten gestionar la autenticación y autorización de los distintos perfiles existentes.

Gestión deportiva

- centro_deportivo
- pista
- deporte
- pista_deporte

Permiten modelar la organización física de las instalaciones deportivas.

Gestión horaria

- plantilla_horaria
- tramo_horario

Definen los horarios disponibles para realizar reservas.

Gestión de reservas

- reserva
- reserva_usuario
- anotacion

Permiten almacenar reservas, participantes y resultados deportivos.

Gestión de profesores

- reserva_fija
- excepcion_reserva_fija
- reserva_extra_clase

Permiten gestionar clases periódicas, liberaciones puntuales y recuperaciones.

Diagrama de secuencia: Inicio de sesión

El proceso de autenticación sigue la siguiente secuencia:

1. El usuario introduce correo electrónico y contraseña.
2. El frontend envía una petición POST al endpoint login.php.
3. El backend consulta la base de datos.
4. Se validan las credenciales.
5. Se recuperan los roles asociados.
6. Se devuelve una respuesta JSON.
7. El frontend almacena los datos del usuario en LocalStorage.
8. Se redirige al dashboard principal.

Diagrama de secuencia: Creación de reserva

El proceso de creación de una reserva sigue los siguientes pasos:

1. El usuario consulta la disponibilidad.
2. El frontend solicita los tramos libres al endpoint disponibilidad.php.
3. El backend calcula los horarios disponibles.
4. El usuario selecciona un tramo libre.
5. El frontend envía una petición al endpoint crear_reserva.php.
6. El backend valida la disponibilidad.
7. Se registra la reserva.
8. Se registra al usuario como participante.
9. Se devuelve una respuesta de confirmación.
10. El frontend actualiza la interfaz.

Principales endpoints desarrollados

La aplicación implementa una API REST compuesta por múltiples endpoints.

Autenticación

- login.php
- registro.php

Consulta de información

- centros.php
- deportes.php
- pistas.php
- disponibilidad.php

Reservas

- crear_reserva.php
- cancelar_reserva.php
- mis_reservas.php

Acompañantes

- agregar_acompanante.php
- quitar_acompanante.php

Anotaciones

- crear_anotacion.php

Gestión de profesores

- reservas_fijas.php
- crear_reserva_fija.php
- liberar_reserva_fija.php
- reserva_extra_clase.php
- fechas_reserva_fija.php
- detalle_reserva_fija.php
- cancelar_reserva_extra_clase.php

Administración

- admin_usuarios.php
- cambiar_estado_usuario.php
- desactivar_reserva_fija.php

3.3. Pruebas realizadas

Durante el desarrollo se realizaron pruebas funcionales utilizando principalmente Postman y pruebas manuales desde la interfaz web.

Prueba 1 – Inicio de sesión

Componente

login.php

Datos de entrada

```
{  
  "email": "profesor1@test.com",  
  "password": "123456"  
}
```

Resultado esperado

Autenticación correcta y recuperación de roles.

Resultado obtenido

Inicio de sesión correcto y acceso al dashboard.

Herramienta utilizada

Postman.

Prueba 2 – Consulta de disponibilidad

Componente

disponibilidad.php

Datos de entrada

id_pista	=	1
fecha	=	2026-06-01

Resultado esperado

Obtención de los tramos disponibles para la fecha seleccionada.

Proyecto “Desarrollo de Aplicaciones Web”



Resultado obtenido

Listado correcto de tramos libres y ocupados.

Herramienta utilizada

Postman y navegador web.

Prueba 3 – Creación de reserva

Componente

crear_reserva.php

Datos de entrada

Reserva válida sobre un tramo libre.

Resultado esperado

Creación de la reserva y asociación del usuario.

Resultado obtenido

Reserva registrada correctamente en la base de datos.

Herramienta utilizada

Interfaz web y Postman.

Prueba 4 – Liberación de clases

Componente

liberar_reserva_fija.php

Datos de entrada

Clase periódica activa.

Resultado esperado

Creación de una excepción para la fecha seleccionada.

Resultado obtenido

La clase queda liberada correctamente y desaparece de la disponibilidad.

Proyecto “Desarrollo de Aplicaciones Web”



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

Herramienta utilizada

Interfaz web.

Prueba 5 – Creación de recuperación

Componente

reserva_extra_clase.php

Datos de entrada

Clase periódica existente.

Resultado esperado

Registro de una recuperación asociada a la reserva fija.

Resultado obtenido

Recuperación creada correctamente y visible en el detalle de la reserva fija.

Herramienta utilizada

Interfaz web y Postman.

4. Proceso de despliegue

4.1 Requisitos software

Para ejecutar la aplicación PistaLibre es necesario disponer de un entorno que permita ejecutar aplicaciones web desarrolladas en PHP y una base de datos MySQL.

El entorno utilizado durante el desarrollo del proyecto ha sido el siguiente:

Software	Versión
Ubuntu	22.04.5 LTS
Apache	2.4.52
PHP	8.1.2
MySQL	8.0.45
Visual Studio Code	Última versión estable
GitHub	Repositorio remoto
Postman	Última versión estable

Adicionalmente, para el despliegue público del proyecto se utilizó el servicio de hosting InfinityFree, compatible con PHP y MySQL.

4.2 Descarga del proyecto

El código fuente del proyecto se encuentra almacenado en un repositorio GitHub.

Para descargarlo se debe ejecutar:

```
git clone https://github.com/ChemaNunnez/pistalibre.git
```

Una vez descargado, acceder al directorio:

```
cd pistalibre
```

4.3 Configuración de la base de datos

Crear una base de datos denominada:

```
CREATE DATABASE pistalibre;
```

El script SQL correspondiente a la estructura de la base de datos se adjunta junto con este manual con el nombre pistalibre.sql.

Las tablas principales utilizadas son:

- usuario
- rol
- usuario_rol
- centro_deportivo
- pista
- deporte
- pista_deporte
- plantilla_horaria
- tramo_horario
- reserva
- reserva_usuario
- anotacion
- reserva_fija
- excepcion_reserva_fija
- reserva_extra_clase

4.4 Configuración de la conexión

Crear el archivo:

```
backend/config/conexion.php
```

con los parámetros de conexión correspondientes al servidor local.

Ejemplo:

```
$host = "localhost";  
$dbname = "pistalibre";  
$usuario = "root";  
$password = "";
```

4.5 Despliegue en Apache

Copiar el proyecto dentro del directorio web de Apache:

```
/var/www/html/
```

La estructura resultante será:

```
/var/www/html/pistalibre
```

Verificar que Apache se encuentra iniciado:

```
sudo systemctl status apache2
```

Si fuera necesario:

```
sudo systemctl start apache2
```

4.6 Acceso a la aplicación

Una vez configurado el entorno, la aplicación estará disponible mediante:

Acceso local:

<http://localhost/pistalibre/frontend/pages/login.html>

Acceso público:

<https://pistalibre.infinityfreeapp.com>

4.7 Usuarios de prueba

Para realizar pruebas funcionales se utilizaron distintos perfiles de usuario.

Administrador

Email: administrador1@test.com

Contraseña: admin123

Proyecto “Desarrollo de Aplicaciones Web”



Profesor

Email: profesor1@test.com

Contraseña: 123456

Usuario

Email: registro2@test.com

Contraseña: 123456

4.8 Hosting

Además del entorno local de desarrollo, la aplicación ha sido desplegada en un entorno de hosting web mediante el servicio InfinityFree.

Características principales del hosting:

- Soporte para PHP.
- Base de datos MySQL.
- Acceso mediante navegador web.
- Gestión de archivos mediante panel web.

URL pública del proyecto:

<https://pistalibre.infinityfreeapp.com>

El despliegue se realizó importando la base de datos MySQL y publicando los directorios backend y frontend dentro del directorio htdocs proporcionado por el hosting.

5. Propuestas de mejoras

Aunque la aplicación cumple los requisitos establecidos inicialmente, existen diversas funcionalidades que podrían implementarse en futuras versiones.

Calendario visual

Implementar una vista de calendario semanal y/o mensual que permita consultar la disponibilidad de las pistas de una forma más intuitiva.

Notificaciones automáticas

Enviar correos electrónicos cuando:

- Se crea una reserva.
- Se cancela una reserva.
- Se libera una clase.
- Se programa una recuperación.

Gestión avanzada de centros

Permitir que cada centro deportivo disponga de:

- Horarios independientes.
- Configuración propia.
- Información de contacto específica.

Gestión de pagos

Integrar pasarelas de pago para realizar reservas de forma online.

Estadísticas deportivas

Incorporar paneles de estadísticas para:

- Usuarios.
- Profesores.
- Administradores.

Aplicación móvil

Desarrollar una versión móvil mediante tecnologías híbridas o nativas.

Sistema de recuperación de contraseña

Permitir a los usuarios restablecer su contraseña mediante correo electrónico.

6. Bibliografía

Apuntes del ciclo formativo

- Apuntes de Desarrollo Web en Entorno Cliente (DWEK).
- Apuntes de Desarrollo Web en Entorno Servidor (DWES).
- Apuntes de Diseño de Interfaces Web (DIW).
- Apuntes de Despliegue de Aplicaciones Web (DAW).

Documentación oficial

- [PHP Documentation](#)
- [MySQL Documentation](#)
- [Apache HTTP Server Documentation](#)
- [MDN Web Docs](#)
- [Git Documentation](#)
- [GitHub Documentation](#)

Herramientas utilizadas

- [Visual Studio Code](#)
- [Postman](#)
- [dbdiagram.io](#)
- [Draw.io \(diagrams.net\)](#)